

Opportunity App

A centralized platform that connects New Jersey students with volunteer opportunities.

Description

The Opportunity App is a web platform built to bridge the gap between students seeking volunteer and internship experiences and the organizations that offer them. High school students across New Jersey can be required to complete volunteer hours to graduate, but finding meaningful opportunities is fragmented across dozens of websites, email chains, and bulletin boards — this app centralizes discovery, application, and communication in one place.

Students can browse and filter opportunities by type, location, skills required, and duration. They can follow organizations, apply directly through the platform, track their application status, send follow-up reminders to organizations, and log completed volunteer experiences on a personal profile. Organizations can post opportunities, manage applicants, send messages, and view impact metrics on their profile page.

The platform supports three user roles — student, organization, and administrator — each with a tailored dashboard and permissions. Administrators can review reported content and manage the opportunity approval queue. All three roles authenticate via email/password or Google Sign-In through the OAuth 2.0 flow.

Team Members

All team members contributed as full-stack developers, taking on backend, frontend, QA, and scrum responsibilities throughout the project.

Name	GitHub	Primary Role
Ryan DeVita	@modestmag-dev	Full-stack (Frontend focus) — Responsible for UI design across the login workflow and the role-based dashboard.
Nguyen Tin Tin Do	@ndo1	Full-stack (Backend focus) — Responsible for the AI-assisted messaging system and FAQ features.
Dev Nitinkumar Hirpara	@devhirpara29	Full-stack — Responsible for the student and organization home dashboards and the follow organizations feature.
Brandon Jachera	@bjach04	Full-stack — Responsible for opportunity search and filtering, home page UI redesign, and Google OAuth integration.
Maximillian Juliano	@mjuliano328	Full-stack (Backend focus) — Responsible for volunteer messaging, opportunity management, and report content features.

Name	GitHub	Primary Role
Nakiwe McDonald	@nakiwem	Full-stack — Responsible for user profiles, application tracking, and the Contact Us feature.
Puzi Wei	@pwei454	Full-stack (Backend focus) — Responsible for the organization dashboard, application notifications, and remind organization feature.

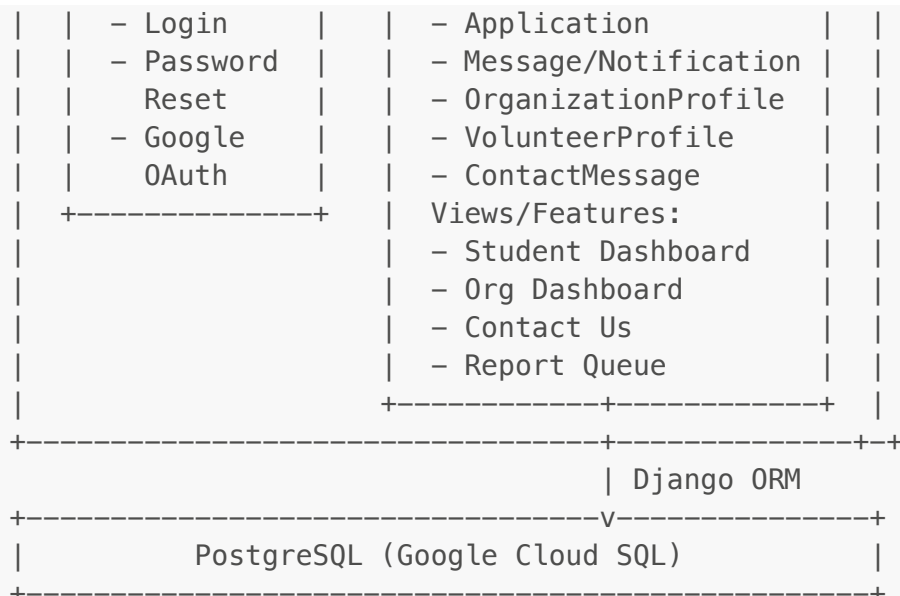
Tech Stack

Layer	Technology
Backend Language	Python 3.12
Backend Framework	Django 5.2.12 + Django REST Framework
Frontend	React 18.3.1 + Vite 5.4.10 + Plain CSS
Database	PostgreSQL (Google Cloud SQL)
Auth	django-allauth 65.7.0 (email + Google OAuth 2.0)
DB Driver	psycopg2-binary 2.9.11
Config	python-dotenv 1.2.2
Frontend Testing	Vitest 3.2.4 + React Testing Library
Backend Testing	Django TestCase + coverage.py
Test DB	SQLite (auto-selected during <code>manage.py test</code>)
Deployment	Google Cloud (Cloud SQL)

Architecture Overview

The app follows Django's MVT (Model-View-Template) architecture. The `accounts` app handles authentication, registration, and password reset. The `pages` app holds all core business logic — models, views, forms, and templates, including role-specific dashboards for students and organizations, and a Contact Us page for user support.





Getting Started

Prerequisites

- Python 3.12+
- pip
- Access to the shared PostgreSQL instance (via `.env`) — contact a team member for credentials

Installation

```
# 1. Clone the repository
git clone https://github.com/drew-csci/opportunity_app_srping_2026.git
cd opportunity_app_srping_2026

# 2. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate          # macOS / Linux
# venv\Scripts\activate          # Windows

# 3. Install dependencies
pip install -r requirements.txt
```

Configuration

Create a `.env` file in the project root (same directory as `manage.py`):

```
DJANGO_SECRET_KEY=dev-insecure-change-me
DJANGO_DEBUG=True
DB_NAME=opportunity_db
DB_USER=oppo_app
DB_PASSWORD=your_db_password
```

```
DB_HOST=34.16.174.60
DB_PORT=5432
ALLOWED_HOSTS=127.0.0.1,localhost
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
```

Never commit `.env` to version control. Contact a team member for the actual credential values.

Run the App

```
python manage.py migrate
python manage.py runserver
```

Visit <http://127.0.0.1:8000>

Usage

1. Browse and apply to opportunities (Student)

```
python manage.py runserver
# Visit http://127.0.0.1:8000/screen1/
```

Log in as a student, use the search bar and filters to find opportunities by keyword, location, or type, click a listing to view details, and submit an application with a cover message. Track all submissions at </my-applications/>.

2. Post and manage opportunities (Organization)

```
python manage.py runserver
# Visit http://127.0.0.1:8000/org/dashboard/
```

Log in as an organization, click **Post Opportunity** to fill in the title, description, required skills, location, and deadline. Review incoming applications, accept or decline applicants, and mark volunteers as complete from the dashboard.

3. Send a message with AI-assisted suggestions (Student)

```
python manage.py runserver
# Visit http://127.0.0.1:8000/organization/<id>/
```

Navigate to an organization's profile page and click **Send Message**. The app generates FAQ suggestions relevant to the opportunity — select one to auto-fill your message or write your own, then submit.

Testing

Backend Tests

Tests use Django's built-in test runner. The test database automatically switches to SQLite when `test` is in `sys.argv` — no extra configuration needed.

```
# Run the full test suite
python manage.py test

# Run only the pages app tests
python manage.py test pages

# Run with full output
python manage.py test --verbosity=2

# Run with coverage report
coverage run manage.py test
coverage report
```

The suite in `pages/tests.py` covers:

- **Unit tests** — individual model methods and form validation
- **Integration tests** — view-to-database flows for application submission, messaging, and profile editing
- **Regression tests** — guard rails around previously fixed bugs (e.g., reminder status check)
- **Smoke tests** — quick pass/fail checks that key pages load without errors
- **Negative/edge-case tests** — invalid inputs, unauthorized access, boundary conditions

Test framework: Django's `unittest`-based `TestCase` and `Client`, with `coverage.py` for coverage reporting.

Frontend Tests

```
npx vitest
```

Project Management

GitHub Project Board: [Opportunity App Spring 2026 Project](#)

All feature work was done on dedicated branches and merged into `main` through pull requests. Each PR required at least one reviewer approval before merging.

- Branch naming: `feature/issue-<number>-short-description` or `<author>_<feature>`
- Most PRs were linked to GitHub Issues and user stories on the project board
- Merge conflicts were resolved on the feature branch before requesting review

Contributors

Contributor	GitHub Profile
Ryan DeVita	github.com/modestmag-dev
Nguyen Tin Tin Do	github.com/ndo1
Dev Nitinkumar Hirpara	github.com/devhirpara29
Brandon Jachera	github.com/bjach04
Maximillian Juliano	github.com/mjuliano328
Nakiwe McDonald	github.com/nakiwem
Puzi Wei	github.com/pwei454

Acknowledgments

Third-Party Libraries and Tools

- Django — web framework
- Django REST Framework — REST API layer
- React — frontend UI library
- Vite — frontend build tool
- django-allauth — authentication including Google OAuth 2.0
- psycopg2 — PostgreSQL adapter for Python
- python-dotenv — environment variable management
- Google Cloud SQL — hosted PostgreSQL database
- Vitest — frontend unit testing framework
- coverage.py — backend test coverage reporting

AI Tools

- **Claude Code (Anthropic)** — used by team members for debugging merge conflicts, writing test scaffolding, and reviewing code during development. All AI-generated suggestions were reviewed, tested, and integrated by the responsible team member.
- **GitHub Copilot** — used by some team members for code completion and boilerplate generation. All output was reviewed before inclusion.

AI tools assisted with development tasks. The design decisions, feature implementations, and work reflected in each team member's PRs represent their own meaningful contributions.